DVC pipeline for phoneme ASR robustness to noise

Yiwei ZUO
22400141

The project repository is available on my GitHub:
`https://github.com/Yiwei-ZUO/phoneme_noise_asr.git`
This report describes the construction of a fully reproducible DVC pipeline for evaluating phoneme ASR robustness to noise. The pipeline is organized into 6 stages. For each stage, I describe the key steps, the test commands used, and the corresponding outputs. Script filenames are highlighted in `blue`. For implementation details, please refer to the GitHub repository.

Pipeline Initialization

First, I created a local project directory using `pixi` and initialized both Git and DVC, then set up the directory structure for *raw data*, *manifests*, *noisy audio*, *predictions*, *results*, and *scripts*. Audio files and large intermediate artefacts were excluded from version control via `.gitignore`.

```
# Create and access the project locally using pixi
pixi init phoneme_noise_asr
cd phoneme_noise_asr

pixi add python
pixi add --pypi dvc

# Initialize Git and DVC
git init
pixi run dvc init

# Create directory structure
mkdir -p data/raw/en/wav
mkdir -p data/manifests/en
mkdir -p data/noisy/en
mkdir -p data/preds/en
mkdir -p results/metrics
mkdir -p results/figures
mkdir -p scripts

# Create configuration file
touch params.yaml dvc.yaml README.md

# Ensure that audio files and large intermediate artefacts are not versioned
echo "data/raw/" >> .gitignore
echo "data/noisy/" >> .gitignore
echo "data/preds/" >> .gitignore
echo "data/manifests/" >> .gitignore
echo ".DS_Store" >> .gitignore # Ignore macOS system files

```

And I committed the initial structure:

```
git add .
git commit -m "initialize project with git and dvc"
```

Then I created a public GitHub repository (**phoneme_noise_asr**) and pushed the local repository:

```
git remote add origin https://github.com/Yiwei-ZUO/phoneme_noise_asr.git
git push -u origin main
```

Stage 0: Download dataset

Since the full Common Voice dataset is too large to download, I used a small subset from the ***Common Voice Spontaneous Speech 1.0 English dataset*** (128 MB) to run the complete pipeline and verify that all stages execute correctly and produce the expected metrics and figures.

To prepare the data, I wrote download_data.py into `scripts`, which randomly samples 100 audio files from `.mp3` to 16kHz mono `.wav` format using `ffmpeg`, and extracts the corresponding transcriptions from `ss-corpus-en.tsv` into `data/raw/en/transcripts.tsv`.

The following commands run this script and test whether the data preparation step executes correctly. Note that `--corpus_dir` should be replaced with the actual path to the users local corpus directory:

```
pixi run python scripts/download_data.py \
  --corpus_dir '/Users/yiwei/CL_M1/CL_M1_S2/Professional Skills/LAB3/sps-corpus-1.0-2025-11-25-en'
```

```
# check the number of wav.
ls data/raw/en/wav/ | wc -l
```

```
100
```

```
# Check the first 3 lines of the transcribed file
head -3 data/raw/en/transcripts.tsv
```

```
spontaneous-speech-en-63608 Well, I would lay on my bed and do nothing until I get better, or until
↪  I feel frustrated because I've been doing nothing and laying in bed all day.

spontaneous-speech-en-63056 I would usually ask them to take a deep breath, uh, and I would I would
↪  assure them that they've got what it takes. Uh, if if they're too nervous, then maybe I would
↪  crack a joke.

spontaneous-speech-en-63489 I think climate change is real and a problem. I don't think it's a
↪  really significantly problem we have to deal with. I have to balance priorities.s
```

Stage 1: Prepare clean manifest

Step (1): I wrote make_manifest.py:

This script generates the clean manifest `data/manifests/en/clean.jsonl`. Rather than scanning folders, it is driven by `transcripts.tsv` to locate audio files, reads the audio metadata, converts the reference text to phoneme sequences using `espeak-ng`, and computes an MD5 checksum for each audio file to ensure traceability. The manifest is written atomically to avoid partial writes.

To verify that the manifest was generated correctly, I checked the number of lines and inspected the first two entries:

```
pixi run python scripts/make_manifest.py \
  --wav-dir data/raw/en/wav \
```

```
  --transcripts data/raw/en/transcripts.tsv \
  --lang en \
  --output data/manifests/en/clean.jsonl
```

```
# See how many rows were generated.
wc -l data/manifests/en/clean.jsonl
```

```
92 data/manifests/en/clean.jsonl
```

I sampled 100 utterances, of which 92 were retained after filtering out sentences containing non-ASCII characters (English only) or special tokens (e.g., `<laugh>`), because I found that the spontaneous dataset occasionally includes non-English sentences, resulting in 92 clean utterances.

```
# Visualize the first two lines
head -2 data/manifests/en/clean.jsonl
```

```
{"utt_id": "en_spontaneous-speech-en-20248",\
 "lang": "en",\
 "wav_path": "data/raw/en/wav/spontaneous-speech-en-20248.wav",\
 "ref_text": "I use my phone, my mobile phone very frequently",\
 "ref_phon": "aɪ jˈuːz maɪ fˈəʊn maɪ mˈəʊbaɪl fˈəʊn vˈɛɹi fɹˈiːkwəntli",\
 "sr": 16000,\
 "duration_s": 10.476,\
 "snr_db": null,\
 "audio_md5": "16ac266723ba27999e62a82cf9f9eddf"}

{"utt_id": "en_spontaneous-speech-en-31014",\
 "lang": "en",\
 "wav_path": "data/raw/en/wav/spontaneous-speech-en-31014.wav",\
 "ref_text": "...celebrate with cake and friends and family",\
 "ref_phon": "sˈɛləbɹˌeɪt wɪð kˈeɪk and fɹˈɛndz and fˈamɪli",\
 "sr": 16000,\
 "duration_s": 6.156,\
 "snr_db": null,\
 "audio_md5": "c719520a0d1c1a526a9f418669e73640"}
```

Step (2): Register this stage in `dvc.yaml` with its dependencies and outputs.
Step (3): Add the file to the Git repository and commit it as Stage 1

```
git add dvc.yaml dvc.lock params.yaml pixi.toml pixi.lock scripts/
git commit -m "add stage 1: make_manifest with parameterized dvc pipeline"
git push
```

Stage 2: Add noise at 10 SNR levels

Step (1): I wrote `add_noise.py` to add noise to each audio file at a specified SNR level ranging from -5 dB to 40 dB. I used the `add_noise` and `add_noise_to_file` functions provided in the lab instructions. A stable MD5-based seed is computed from each `utt_id` and `snr_db` to ensure reproducibility. The script reads the clean manifest, writes noisy audio files to `data/noisy/en/{snr}db/`, and generates a new manifest with updated `wav_path` and `snr_db` fields.

I tested the script with SNR = 10 dB and confirmed that 92 utterances were generated successfully by the following codes:

```
pixi run python scripts/add_noise.py \
```

```
  --clean-manifest data/manifests/en/clean.jsonl \
  --lang en \
  --snr-db 10 \
  --output-dir data/noisy/en/10db \
  --output-manifest data/manifests/en/noisy_10db.jsonl
```

```
Successfully created noisy manifest: data/manifests/en/noisy_10db.jsonl
```

```
# Check how many records were generated
wc -l data/manifests/en/noisy_10db.jsonl
```

```
92 data/manifests/en/noisy_10db.jsonl
```

```
# Looking at the first line, we can confirm that the snr_db field has become 10
head -1 data/manifests/en/noisy_10db.jsonl
```


```
{"utt_id": "en_spontaneous-speech-en-20248",\
 "lang": "en",\
 "wav_path": "data/noisy/en/10db/spontaneous-speech-en-20248.wav",\
 "ref_text": "I use my phone, my mobile phone very frequently",\
 "ref_phon": "aɪ jˈuːz maɪ fˈəʊn maɪ mˈəʊbaɪl fˈəʊn vˈɛɹi fɹˈiːkwəntli",\
 "sr": 16000,\
 "duration_s": 10.476,\
 "snr_db": 10.0,\
 "audio_md5": "16ac266723ba27999e62a82cf9f9eddf"}
```


Step (2): Update `params.yaml` to increase to 10 noise levels.
Step (3): Add noise phase to all SNR, update `dvc.yaml`
I test the output (To simplify the report, I have omitted intermediate outputs):

```
pixi run dvc repro
```

```
Running stage 'add_noise@en--5':
> pixi run python scripts/add_noise.py --clean-manifest data/manifests/en/clean.jsonl --lang en
↪  --snr-db -5 --output-dir data/noisy/en/-5db --output-manifest data/manifests/en/noisy_-5db.jsonl
Successfully created noisy manifest: data/manifests/en/noisy_-5db.jsonl
Updating lock file 'dvc.lock'
...
Running stage 'add_noise@en-40':
> pixi run python scripts/add_noise.py --clean-manifest data/manifests/en/clean.jsonl --lang en
↪  --snr-db 40 --output-dir data/noisy/en/40db --output-manifest data/manifests/en/noisy_40db.jsonl
Successfully created noisy manifest: data/manifests/en/noisy_40db.jsonl
```

Step (4): Add the file to the Git repository and commit it as Stage 2.

```
git add dvc.lock dvc.yaml params.yaml scripts/add_noise.py
git commit -m "Stage 2: add noise at 10 SNR levels"
git push
```

Stage 3: Running the wav2vec2 model for phoneme recognition

Step (1): I wrote `scripts/run_asr.py`:
it loads the pre-trained model `facebook/wav2vec2-lv-60-espeak-cv-ft`, reads each audio file from the input manifest, and outputs the predicted phoneme sequence for each utterance.

The script takes a manifest as input and produces a new prediction manifest with an additional `pred_phon` field, following the same manifest-based pipeline design.

I first tested the model on the clean manifest to verify that the stage works correctly:

```
pixi run python scripts/run_asr.py \
--input-manifest data/manifests/en/clean.jsonl \
--output-manifest data/preds/en/preds_clean.jsonl
```

```
[1] en_spontaneous-speech-en-20248 → aɪ j uː z m aɪ f oʊ m aɪ m oʊ b əl p oʊ n v ɛ ɹ i f ɹ ɪ k w ə n
↪  t l i
[2] en_spontaneous-speech-en-31014 → w ʌ z s ɛ l ɪ b ɹ eɪ ɾ w ɪ ð k eɪ k æ n d f ɹ ɛ n d z æ n d f æ
↪  m l i
...
[92] en_spontaneous-speech-en-71306 → tʃ ɐ n eɪ l m ə n t s ʌ m t aɪ m z æ s k m aɪ d ɑː k t ɚ z ɐ l
↪  ɑː ɾ ʌ t aɪ m z aɪ æ s k m ɛ d ɪ k əl dʒ ɜː n əl z aɪ n ɛ v ɚ æ s k w ɛ b ɛ m d iː w ɛ b ɛ m d
↪  iː l h æ v j uː h d aɪ ɪ n ʌ v k æ n s ɚ dʒ ʌ s b ɪ k ʌ z j uː h æ v ɐ n ɪ tʃ ɔ n j ɚ f ʊ t
```

I then verified the output by checking the number of predictions and inspecting the first entry to confirm that the `pred_phon` field was correctly added to each record:

```
wc -l data/preds/en/preds_clean.jsonl
head -1 data/preds/en/preds_clean.jsonl
```

```
92
{"utt_id": "en_spontaneous-speech-en-20248",\
 "lang": "en",\
 "wav_path": "data/raw/en/wav/spontaneous-speech-en-20248.wav",\
 "ref_text": "I use my phone, my mobile phone very frequently",\
 "ref_phon": "aɪ jˈuːz maɪ fˈəʊn maɪ mˈəʊbaɪl fˈəʊn vˈɛɹi fɹˈiːkwəntli",\
 "sr": 16000,\
 "duration_s": 10.476,\
 "snr_db": null,\
 "audio_md5": "16ac266723ba27999e62a82cf9f9eddf",\
 "pred_phon": "aɪ j uː z m aɪ f oʊ m aɪ m oʊ b əl p oʊ n v ɛ ɹ i f ɹ ɪ k w ə n t l i"}
```

Step (2): I registered the `run_asr` stage in `dvc.yaml` using `foreach` over `langs`, with the script reading `snr_levels` directly from `params.yaml`. This means the model is loaded only once per language, processing all SNR levels in a single stage. New languages or noise levels can be added by simply updating `params.yaml` without modifying any code.

Step (3): Add the file to the Git repository and commit it as Stage 3.

```
git add dvc.lock dvc.yaml scripts/run_asr.py
git commit -m "Stage 3: run wav2vec2 ASR on noisy manifests"
git push
```

Stage 4: Evaluation

Step (1): I wrote `evaluate.py`: it computes the Phoneme Error Rate (PER) using edit distance between `ref_phon` and `pred_phon`. Since the two fields use different formats, a normalization step is applied to both sequences before comparison: stress markers, length markers, and diacritics are removed, and each sequence is tokenized character by character (ignoring spaces), treating each IPA character as a single phoneme unit.

Step (2): I used a DVC matrix in `dvc.yaml` to automatically generate evaluation stages for all 10 SNR levels.

```
pixi run dvc repro
```

```
Running stage 'evaluate@en--5':
> pixi run python scripts/evaluate.py --pred-manifest data/preds/en/preds_-5db.jsonl --output
↪  results/metrics/en_-5db.json --lang en --snr-db -5
PER (en, SNR=-5dB): 0.9160
Updating lock file 'dvc.lock'

Running stage 'evaluate@en-0':
> pixi run python scripts/evaluate.py --pred-manifest data/preds/en/preds_0db.jsonl --output
↪  results/metrics/en_0db.json --lang en --snr-db 0
PER (en, SNR=0dB): 0.8690
Updating lock file 'dvc.lock'

...
Running stage 'evaluate@en-40':
> pixi run python scripts/evaluate.py --pred-manifest data/preds/en/preds_40db.jsonl --output
↪  results/metrics/en_40db.json --lang en --snr-db 40
PER (en, SNR=40dB): 0.2502
Updating lock file 'dvc.lock'
```

Step (3): Add the file to the Git repository and commit it as Stage 4.

```
git add dvc.lock dvc.yaml scripts/evaluate.py results/metrics/
git commit -m "Stage 4: compute PER with character-level phoneme normalization"
git push
```

Stage 5: plot results

Step (1): I wrote `plot_results.py`: it reads the metrics directory and constructs file paths based on `params.yaml`, plots PER as a function of SNR level for each language, and computes the cross-language mean curve. The resulting figure is saved atomically to `results/figures/`.

Step (2): I registered the `plot_results` stage in `dvc.yaml`. The script reads `langs` and `snr_levels` directly from `params.yaml`, so no code changes are needed when adding new languages.

```
pixi run python scripts/plot_results.py \
  --metrics-dir results/metrics \
  --output results/figures/per_vs_snr.png \
  --params params.yaml
```

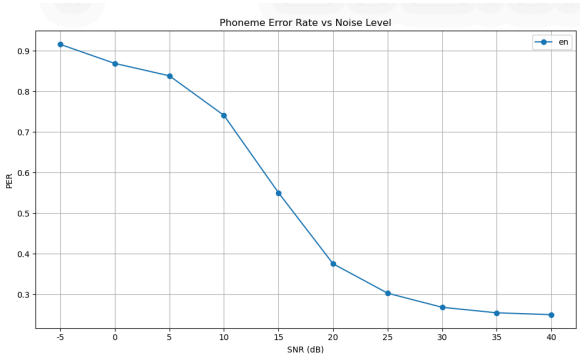


Figure 1: PER vs Noise level in *en*

Analysis: Figure 1 shows that PER decreases consistently as the SNR level increases, indicating that the ASR model performs better when the audio contains less noise. At the lowest SNR level (-5 dB), PER reaches about 0.91, while at the highest SNR (40 dB) it decreases to around 0.25. The curve shows a steep drop between 10 dB and 25 dB, suggesting that the model is particularly sensitive to noise in this range.

Step (3): Add the file to the Git repository and commit it as Stage 5.

```
git add dvc.lock dvc.yaml scripts/plot_results.py results/figures/.gitignore
git commit -m "Stage 5: plot PER vs SNR"
git push
```

Now, I run successfully the complete pipeline on a small dataset (english) and verify that all stages execute correctly and that the expected metrics and figures are produced.

Stage 6: Add another language : *French*

I added French, After download the ***Common Voice Spontaneous Speech 2.0 - French***.
I only updated `langs` in `params.yaml`:

```
langs:
  - en
  - fr
```

Then re-ran the pipeline without modifying any code:

```
pixi run dvc repro
```

The output of the figure is as follows:
(Here: French curves appears, and the mean curve is updated automatically.)

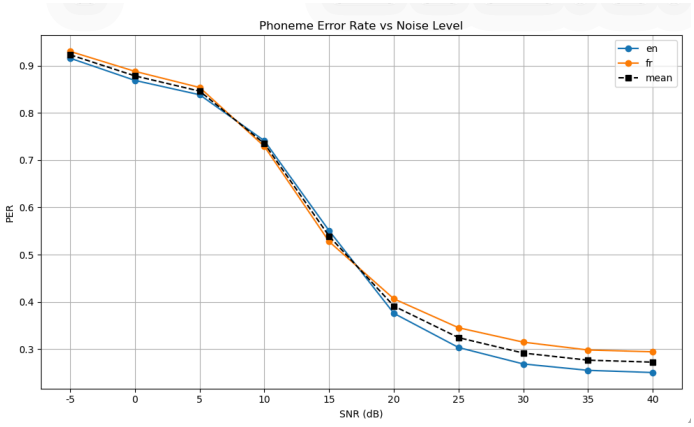


Figure 2: PER vs Noise level for English (en) and French (fr) with cross-language mean

Analysis: Figure 2 shows the PER curves for English and French across 10 SNR levels, along with their cross-language mean. Both languages follow the same trend: PER decreases consistently as SNR increases, confirming that the model performs better when audio contains less noise. At SNR = -5dB, both languages reach a PER of around 0.92, indicating that the model almost completely

fails under heavy noise. As SNR increases, the two curves begin to diverge: English consistently outperforms French, reaching a PER of approximately 0.25 at 40dB compared to 0.30 for French. This gap suggests that the `facebook/wav2vec2-lv-60-espeak-cv-ft` model, despite being trained on multilingual data, remains more robust on English. The steep drop between 10dB and 20dB is observed in both languages, indicating that this SNR range is the critical threshold for model performance.

```
git add .
git commit -m "finalize pipeline with English and French results"
git push
```

Finally, I committed all remaining changes to ensure the GitHub repository reflects the latest state of the pipeline.